Chapter 7

MEMORY ORGANIZATION

Contents:

1. Memory Hierarchy
2. Associative Memory
3. Cache Memory: Cache Mapping techniques
4. Replacing data in cache, writing data to the cache, cache performance basics

7.1 Memory

- A memory stores large number of items commonly referred as words.
- Each word consists of a specific number of bits, for e.g. 8 bits.
- Therefore, a memory can be viewed as:
  - ***m*** words of ***n*** bits
  - each for a total of ***m x n*** bits

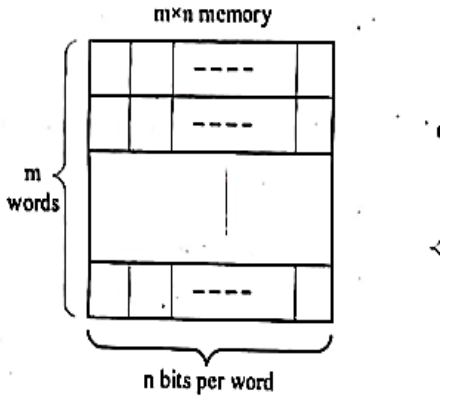


Figure: Words and bits per word in memory

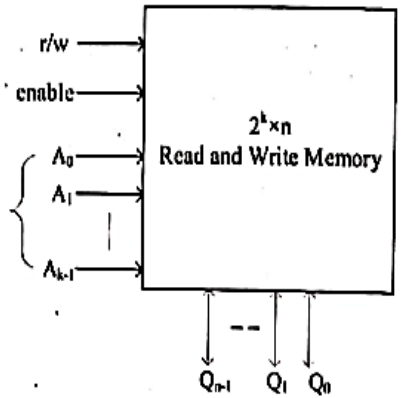
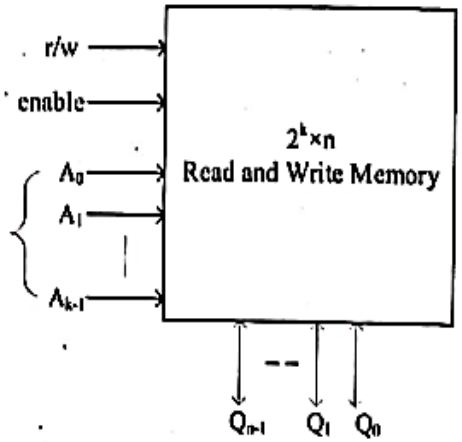


Figure: Memory Block diagram

- If a memory has *k* address inputs, it can have up to 2^k words ($m=2^k$) which implies that; $k=\log_2(m)$.
- This means that $\log_2(m)$ address input signals are required to identify a particular word.
- Also, n data signals are required to output a selected word.
- For example:

  4096 x 8 memory:
  - can store 32768 bits and
  - require 12 address signals (i.e $k= \log_2(4096) = 12$)
  - eight I/O data signals.



7.2 Memory Hierarchy

- There are three key characteristics of memory that can put them in a hierarchical order, namely: capacity, access time, and cost.
- The below relationship holds true for the memories:
  - Faster the access time, the greater is the cost per bit
  - Greater the storage capacity, smaller the cost per bit
  - Greater storage capacity, slower access time

- To meet performance requirements, the ES designer wishes to use expensive, relatively lower-capacity memories with short access times.
- But ultimately, the cost increases.
- So, the designer should employ a memory hierarchy.

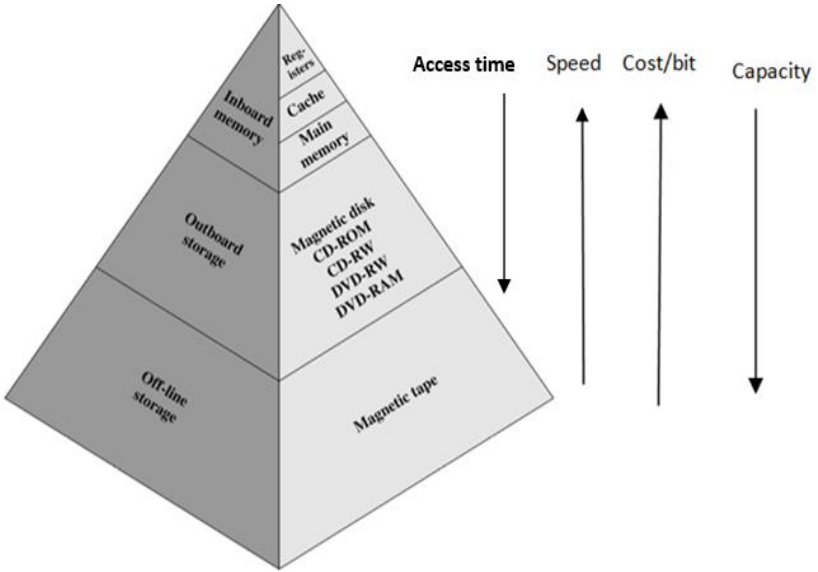


Figure: Memory hierarchy

- As one goes down the hierarchy, the following occur:
  - Decreasing cost per bit
  - Increasing capacity
  - Increasing access time
  - Decreasing frequency of access of the memory by the processor

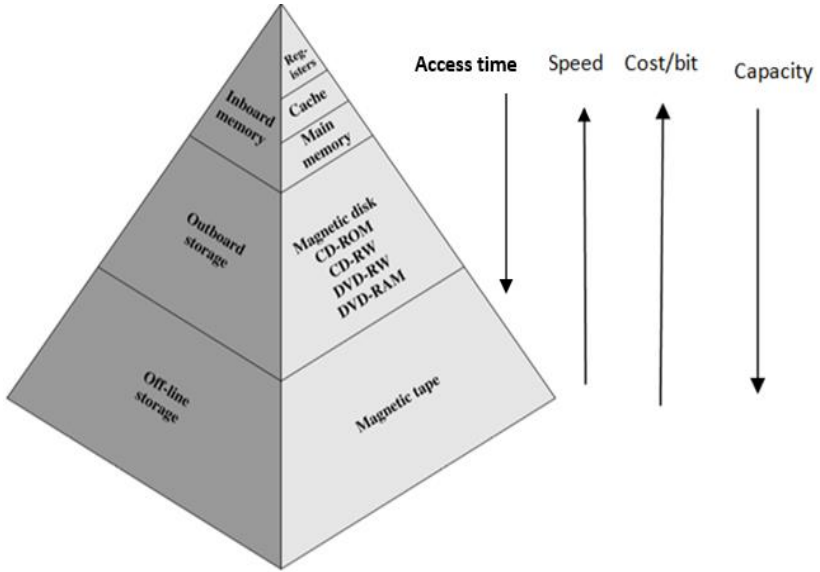


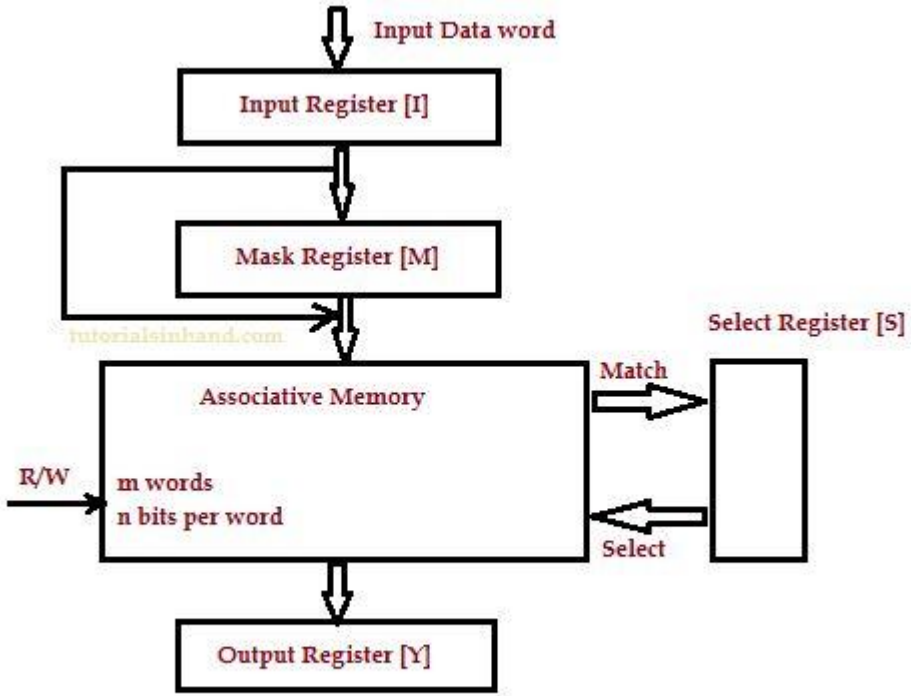
Figure: Memory hierarchy

7.2 Associative Memory

- Aka **Content Addressable Memory** (CAM)
- A memory unit accessed by content, rather than by address, is called CAM.
- When a write operation is performed on associative memory, no address or memory location is given to the word.
  - The memory itself is capable of finding an empty unused location to store the word.
- For read operation, the content of the word, or part of the word, is specified.
  - The words which match the specified content are located by the memory and are marked for reading.

Block diagram of CAM

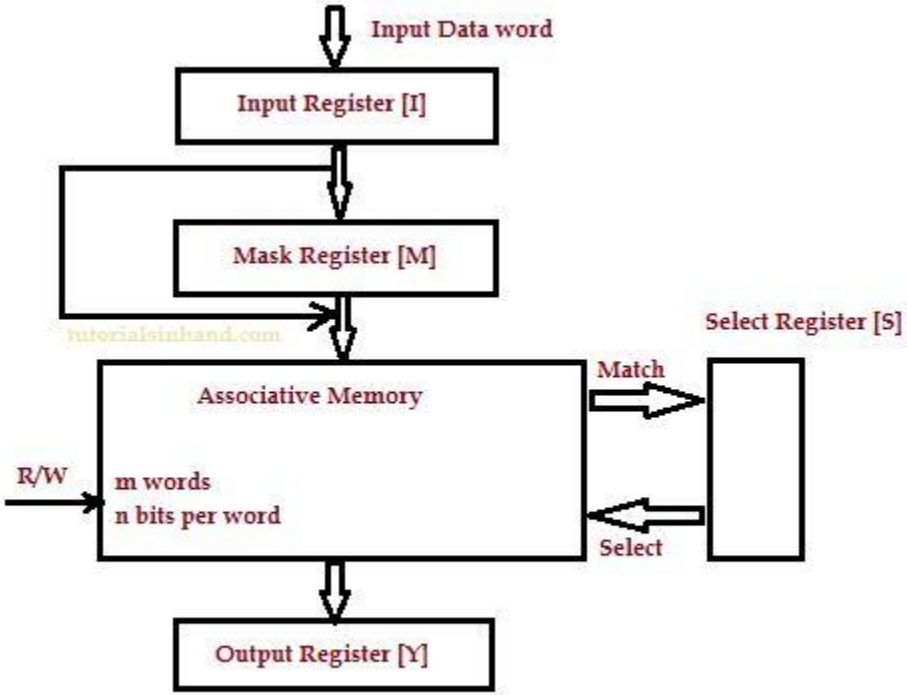
1. **Input Register (I):**
   - hold the data that is to be written into the associative memory.
   - It is also used to hold the data that is to be searched for.
   - At a particular time it can hold a data containing one word of length (say n).
2. **Mask Register (M)**
   - is used to provide a mask for choosing a key or particular field in the input register's word.
   - Since input register can hold a data of one word of length n so the maximum length of mask register can be n.
3. **Select Register (S)**
   - contains m bits, one for each memory words.
   - When input data in I register is compared to key in m register and match is found then that particular bit is set in select register.
4. **Output Register (Y)**
   - contains the matched data word that is retrieved from associative memory.



Block Diagram of Associative Memory

Working of CAM

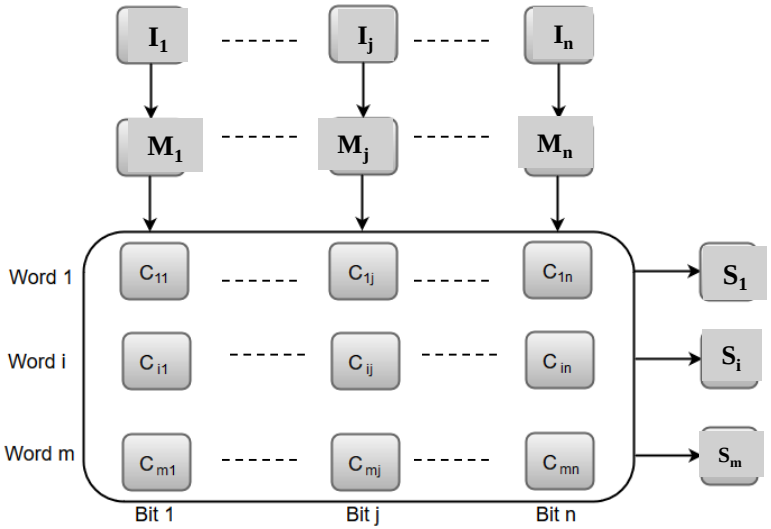
- The words which are kept in the memory are compared in parallel with the content of the Input Register.
- The Mask register (M) provides a mask for choosing a particular field or key in the Input word.
- If the Mask Register contains a binary value of all 1's, then the entire argument is compared with each memory word.
- Otherwise, only those bits in the Input that have 1's in their corresponding position of the Mask register are compared.



Block Diagram of Associative Memory

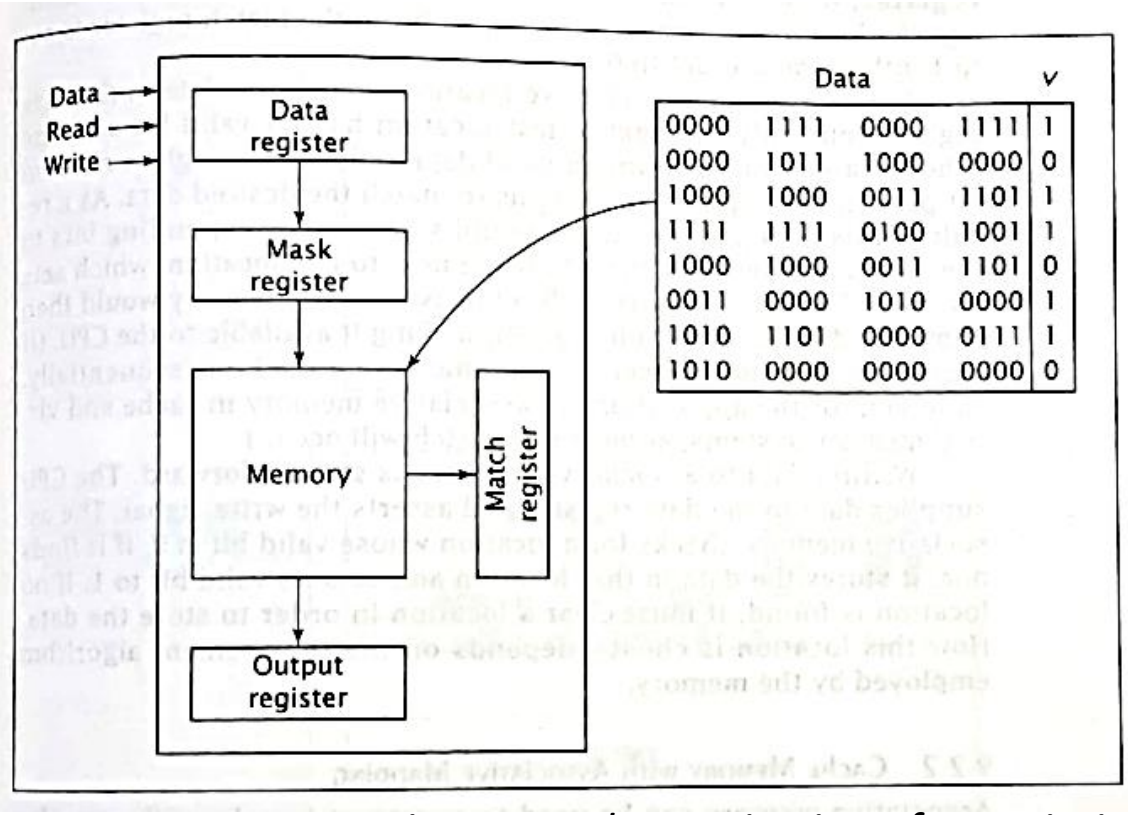
- The cells present inside the memory array are marked by the letter C with two subscripts.
  - The first subscript gives the word number and the second specifies the bit position in the word.
  - For instance, the cell C_{ij} is the cell for bit **j** in word **i**.
- A bit I_j in the Input register is compared with all the bits in column **j** of the array provided that $M_j = 1$.
  - This process is done for all columns **j** = 1, 2, 3....., n.
- If a match occurs between all the unmasked bits of the Input and the bits in word **i**, the corresponding bit S_i in the match register is set to 1.
- If one or more unmasked bits of the Input and the word do not match, S_i is cleared to 0.

Associative memory of m word, n cells per word:



More study:
https://www.youtube.com/watch?v=SV7Kk1njt5c

- The associative memory searches all of its locations in parallel and marks the locations that match the specified data input. The matching data values are then read out sequentially.
- Cache memory is constructed using either static RAM or associative memory, depending on mapping scheme used.
- SRAM receives an address and access the data at that address.
- An associative memory is more expensive than a RAM because each cell must have storage capability as well as logic circuit for matching its contents with an external argument.
- For this reason, associative memories are used in application where search time is very critical and short.



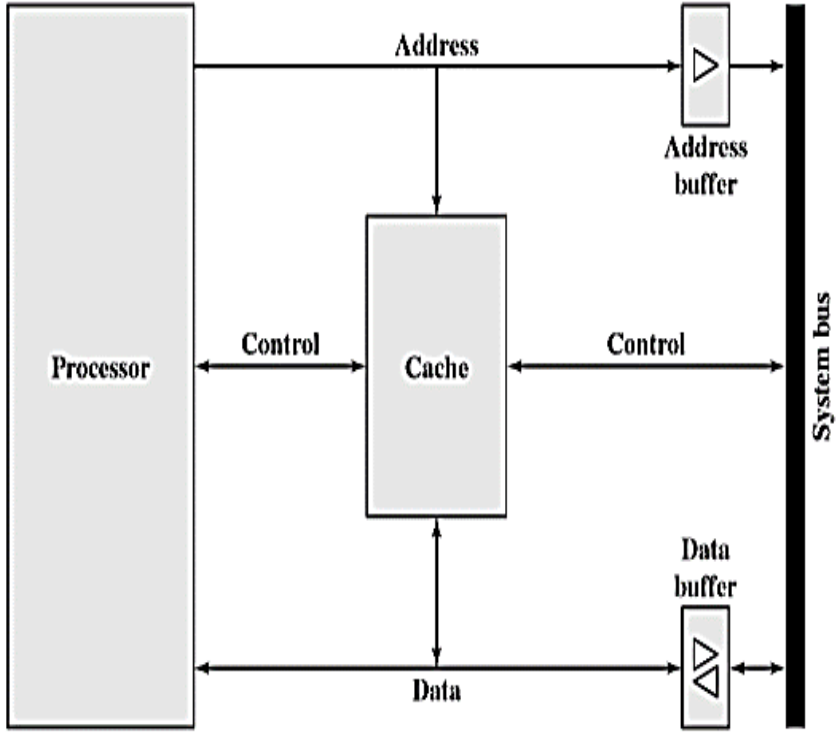
Note: This is book's diagram. Please refer book.

Fig: Internal organization of Associative memory

7.3 Cache

- Cache memory is a small-sized type of volatile computer memory that provides high-speed data access to a processor and stores frequently used computer programs, applications and data.
- It is the fastest memory in a computer, and is typically integrated onto the motherboard and directly embedded in the processor or main random-access memory **(RAM).**
- Cache is usually designed using static RAM rather than dynamic RAM, which is one reason that cache is more expensive but faster than main memory.
- When the processor attempts to read a word of memory, a check is made to determine if the word is in the cache.
  - If so, the word is delivered to the processor (ie. Cache Hit).
  - If not (ie. Cache Miss), a block of main memory, consisting of fixed number of words is read into the cache and then the word is delivered to the processor.

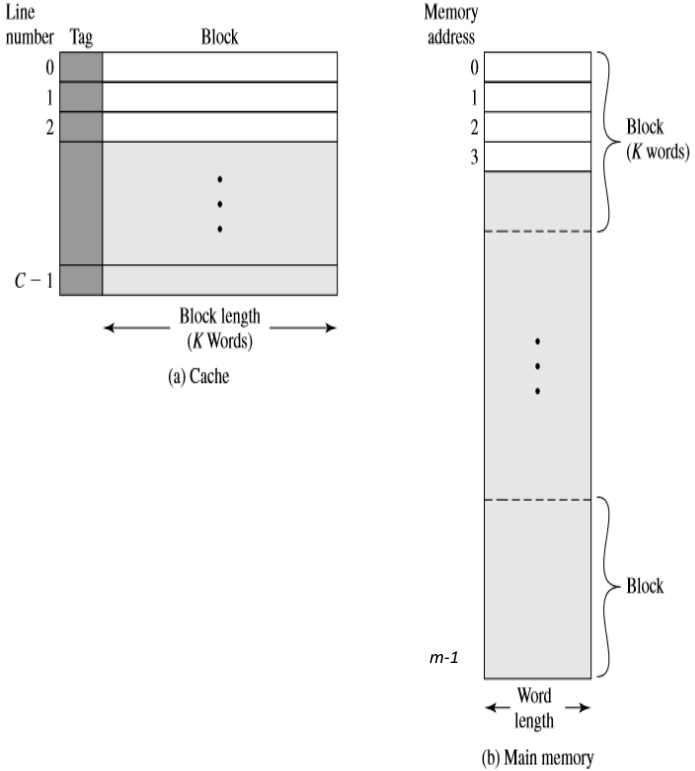
- Cache connects to the processor via data control and address line.
- The data and address lines also attached to data and address buffer which attached to a system bus from which main memory is reached.
- When a cache hit occurs, the data and address buffers are disabled and the communication is only between processor and cache with no system bus traffic.
- When a cache miss occurs, the desired word is first read into the cache and then transferred from cache to processor. For later case, the cache is physically interposed between the processor and main memory for all data, address and control lines.



Typical Cache Organization

Structure of Cache and Main memory:

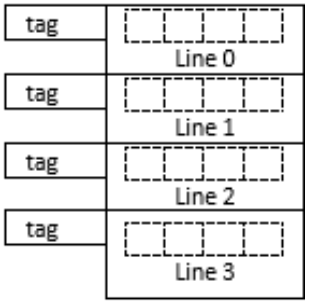
- Memory is represented as m x n.
  - Where m is no of words, and n is no of bits in each word.
  - Memory consists of fixed length word size.
  - Address lines = $\log_2(m)$
  - Group of words is called a block.
  - Each block consists of K words, each being fixed length.
  - No of blocks i.e. B = m/K blocks.
- Cache consists of C lines of K words each.
  - Each line contains K words, plus a tag of a few bits.
  - The tag is usually a portion of the main memory address.
  - The number of lines (C) of cache is less than the number of main memory blocks (B).



Example:

- Consider a 64x8 memory
  - Here, No of words (m) = 64
  - and n is no of bits in each word (n) = 8
  - Address lines = $\log_2(64) = 5$
  - Here, block size = 4
  - No of block = 64/4 = 16

- Consider a cache of 4 lines
  - Each line contains 4 words, plus a tag



cache

Block 0
Block 1
Block 2
Block 3
Block 4
Block 5
Block 6
Block 7
Block 8
Block 9
Block 10
Block 11
Block 12
Block 13
Block 14
Block 15

Memory

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15
16	17	18	19
20	21	22	23
24	25	26	27
28	29	30	31
32	33	34	35
36	37	38	39
40	41	42	43
44	45	46	47
48	49	50	51
52	53	54	55
56	57	58	59
60	61	62	63

words in each block

7.3 Cache Mapping Techniques:

- The transformation of data from main memory to cache memory is referred to as memory mapping process.
- Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines.
- There are three different types of mapping functions in common use and are:
  - direct, associative and set associative.

1. Direct Mapping

- It is the simplest mapping technique.
- It maps certain block of main memory into only one possible cache line, i.e. a given main memory block can be placed in one and only one place on cache.
- It implements one-to-many function, i.e., one cache line can map many blocks of memory.
- The cache line no can be calculated as:

  i = j MOD m

  Where, i = cache line number;
  j = main memory block number;
  m = number of lines in the cache
- For e.g., if there are 16 blocks of main memory and 4 cache lines, and if we need to identify which cache line maps the block no 14 of the main memory.
  - Here, i = 14 MOD 4 =2
  - So, the block no 14 of the main memory is mapped by cache line no 2.
- Here, memory consists of 16 blocks of each containing 4 words in an array (say Block 0 has words 0,1,2,3 and Block 1 has 4,5,6,7….Block 15 has 60,61,62,63).
- Similarly, each cache line maps 4 blocks of memory.

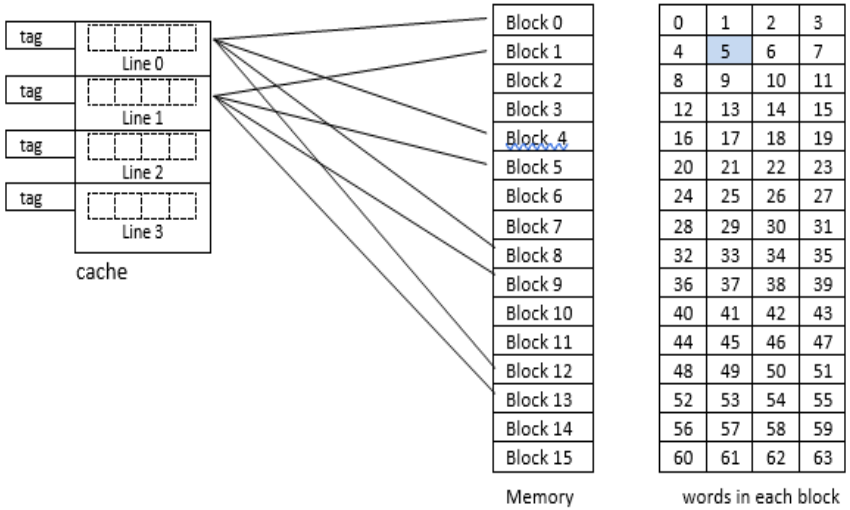


Fig: Direct Cache Mapping

1. Direct Mapping (Contd.)

- Each physical address generated by the CPU is viewed as three fields:

Cache tag bits	Cache line no	Word

- For eg, if CPU generates a 6 bit address 000101, it is viewed as:

Tag bits	Line no	Word
00	01	01

  - This tells to look at cache line no 1 tagged as 00 at its second index if desired address is present.
- So, a **2-bit tag** is enough to identify the block of memory in this case.
- If the search is matched, this is a cache hit. But if the search does not match, then it is a miss.

- **Advantage**: easy to implement
- **Disadvantage**: not flexible, contention problem

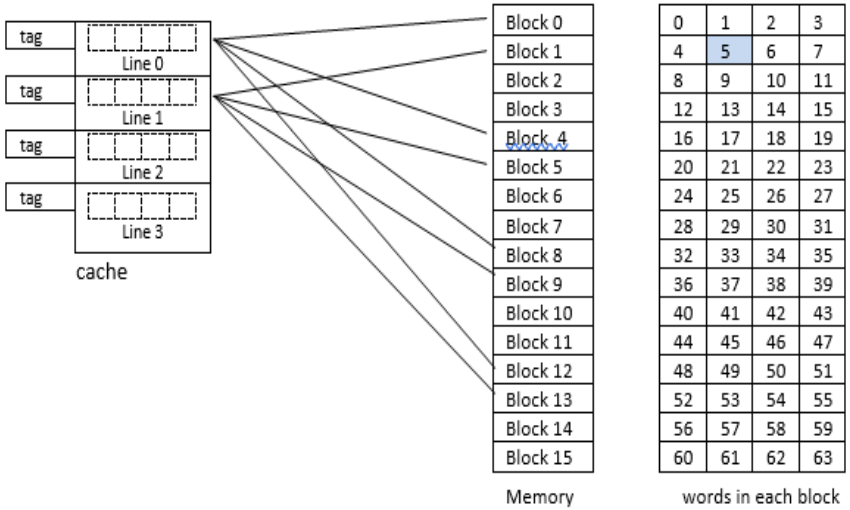


Fig: Direct Cache Mapping

Contention is **what happens when demand for a shared resource exceeds the supply**

2. Associative Mapping

- This is a much more flexible mapping technique.
- Any main memory block can be loaded to any cache line position.
- It implements many-to-many function.
- **Advantage:** flexibility
- **Disadvantage:** increased hardware cost, increased access time
- Each physical address generated by the CPU is viewed as two fields:

Cache tag no	Word

- For eg, if CPU generates a 6-bit address 000101, it is viewed as:

Tag no	word
0001	01

  - This tells to look at cache line tagged as 0001 if it contains the word 01 of the block 0001 of the main memory.
  - In this case, a 4-bit tags are required to identify those 16 blocks of main memory. The tag no represents the block no of the main memory.
- Here, we need to search for all the 16 tags from 0000 to 1111 to determine whether a given block is in the cache or not.
- Therefore, the cost of implementation of hardware (i.e comparator) will be higher.

The figure below represents the associative mapping:

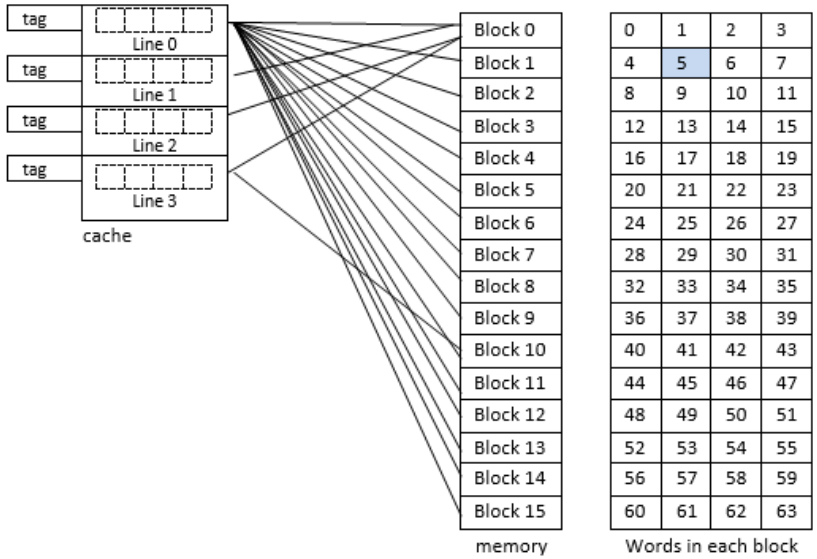


Figure: Associative mapping

Reference:
https://www.youtube.com/watch?v=uwnsMaH-iV0

3. Set-associative Mapping

- This is the combination of the direct and associative technique. Overcomes the limitation of contention and increased hardware cost.
- In this case, lines of the cache are grouped into sets and the mapping allows to reside in any block of a particular set.
- It implements many-to-many function.
- The cache set no can be calculated as:

  $$k = p \text{ MOD } q$$

  Where, k = set number

  p = main memory block number;

  q = number of sets in the cache
- For e.g., if there are 16 blocks of main memory and 2 sets in cache, and if we need to identify which cache set maps the block no 14 of the main memory.
  - Here, k = 14 MOD 2 =0
  - So, the block no 14 of the main memory is mapped by cache set no 0.
  - Hence, Block 0,2,4,6,8,10,12,14 is mapped to set no 0.
  - Other block are mapped to set 1

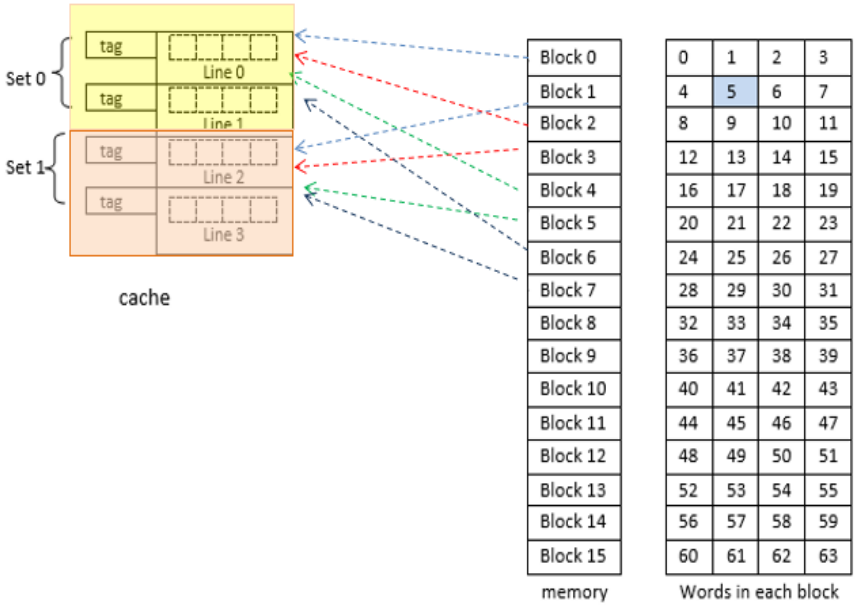


Figure: Set Associative cache mapping

Reference:
https://www.youtube.com/watch?v=KhAh6thw_TI

3. Set-associative Mapping (Contd.)

- Each physical address generated by the CPU is viewed as three fields:

Cache tag no	Set no	Word

- For e.g., if CPU generates a 6-bit address 000101, it is viewed as:

Tag no	Set No	Word
000	1	01

- This tells to look at cache line **tagged as 000** and **set 1** if it contains the **word 01** of the **block 0000** of the main memory.
- First, it compares set no 1 if it is present in SET 0 (i.e from 00 and 01), and then looks for tag no 000 and finally looks for the index 2 of the cache line 1.
- In this case, a 3-bit tags are required to identify those 8 blocks of main memory. The tag no represents the block no of the main memory.

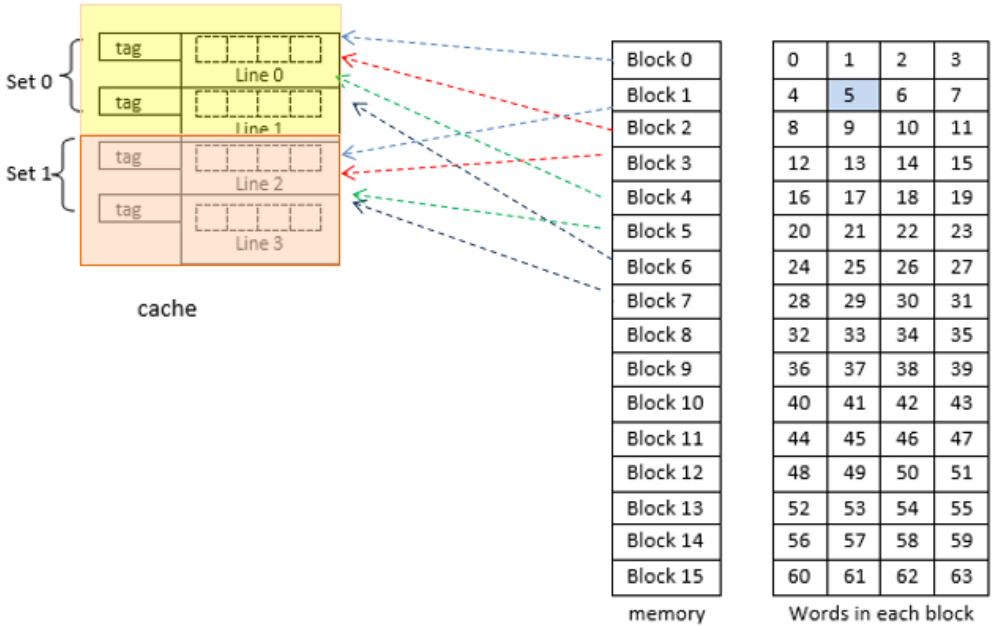


Figure: Set Associative cache mapping

Reference:
https://www.youtube.com/watch?v=KhAh6thw_TI

7.4 Cache Replacement Policy:

- The cache replacement policy is the technique for choosing which cache block to replace when Full Associative cache is full, or when a Set-associative cache's line is full.
- It is to be noted that there is no choice in a direct-mapped cache; a main memory address always maps to the same cache address and thus replaces whatever block is already there.
- There are three common replacement policies:
  - Random
  - Least Recently Used
  - First In First Out

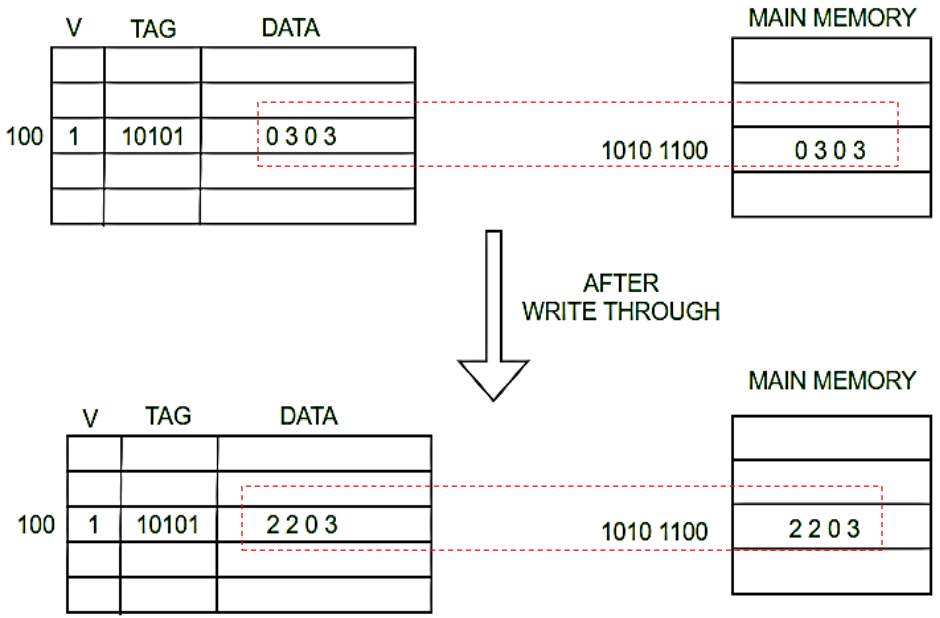
i)	Random	• This policy chooses the block to replace randomly. • It is simple to implement. • However, it does not prevent replacing a block that is likely to be used again soon.
ii)	Least Recently Used (LRU)	• This policy replaces the block that has not been accessed for the longest time, assuming that this means that it is least likely to be accessed in the near future. • This policy provides for an excellent hit/miss ratio but requires expensive hardware to keep track of the times blocks are accessed.
iii)	First In First Out (FIFO)	• Replace that block in the set which has been in the cache longest • Uses a queue of size N, pushing each block address onto the queue when the address is accessed, and then choosing the block to be replaced by popping the queue.

Cache Writing technique/policy:

- Cache is a technique of storing a copy of data temporarily in rapidly accessible storage memory.
- Cache stores most recently used words in small memory to increase the speed at which data is accessed.
- It acts as a buffer between RAM and CPU and thus increases the speed at which data is available to the processor.

a) Write Through

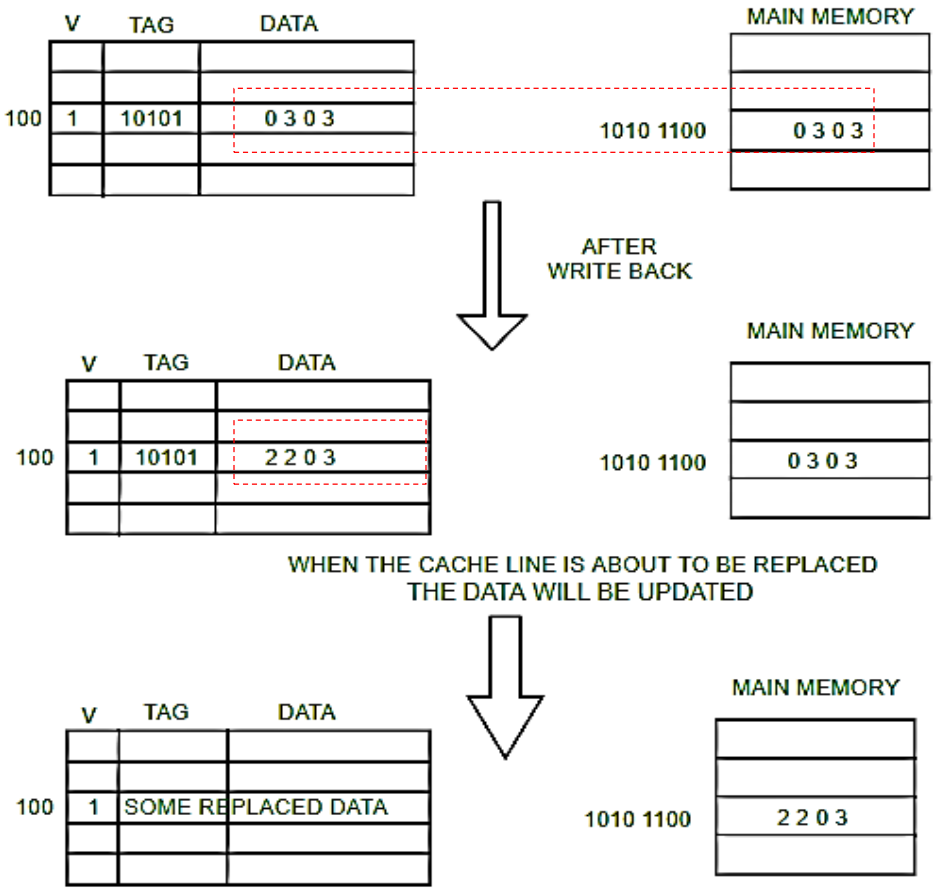
- In write-through, data is simultaneously updated to cache and memory.
- This process is simpler and more reliable, keeps the cache and memory consistent.
- This is used when there are no frequent writes to the cache (The number of write operations is less).
- A Write-through cache colves the inconsistency problem by forcing all writes to update both the cache and the main memory.
- Main issues: Badwidth



Reference:
https://www.youtube.com/watch?v=uM8K0Z5usu8

b) Write Back:

- In this policy, the memory is not updated until the cache block needs to be replaced.
- The data is updated only in the cache and updated into the memory at a later time.
- Data is updated in the memory only when the cache line is ready to be replaced .
- The modified cache content will be first be written to the main memory.
- Only then can the cache lie be replaced with new data.



Exam Questions:

1. What is cache performance? Describe in brief the strategies used to write data to the cache memory. [PU 2017 Fall]
2. What is memory hierarchy? Differentiate between different types of cache mapping. [PU 2017 Spring]
3. What is Memory hierarchy? Describe the significance of Cache [PU 2018 Fall]
4. What is cache memory? Briefly explain Direct and Set-associative mapping. [PU 2019 Spring]
5. What is cache memory? Explain associative, set associative and direct mapping in cache. [PU 2020 Spring]

End of Chapter